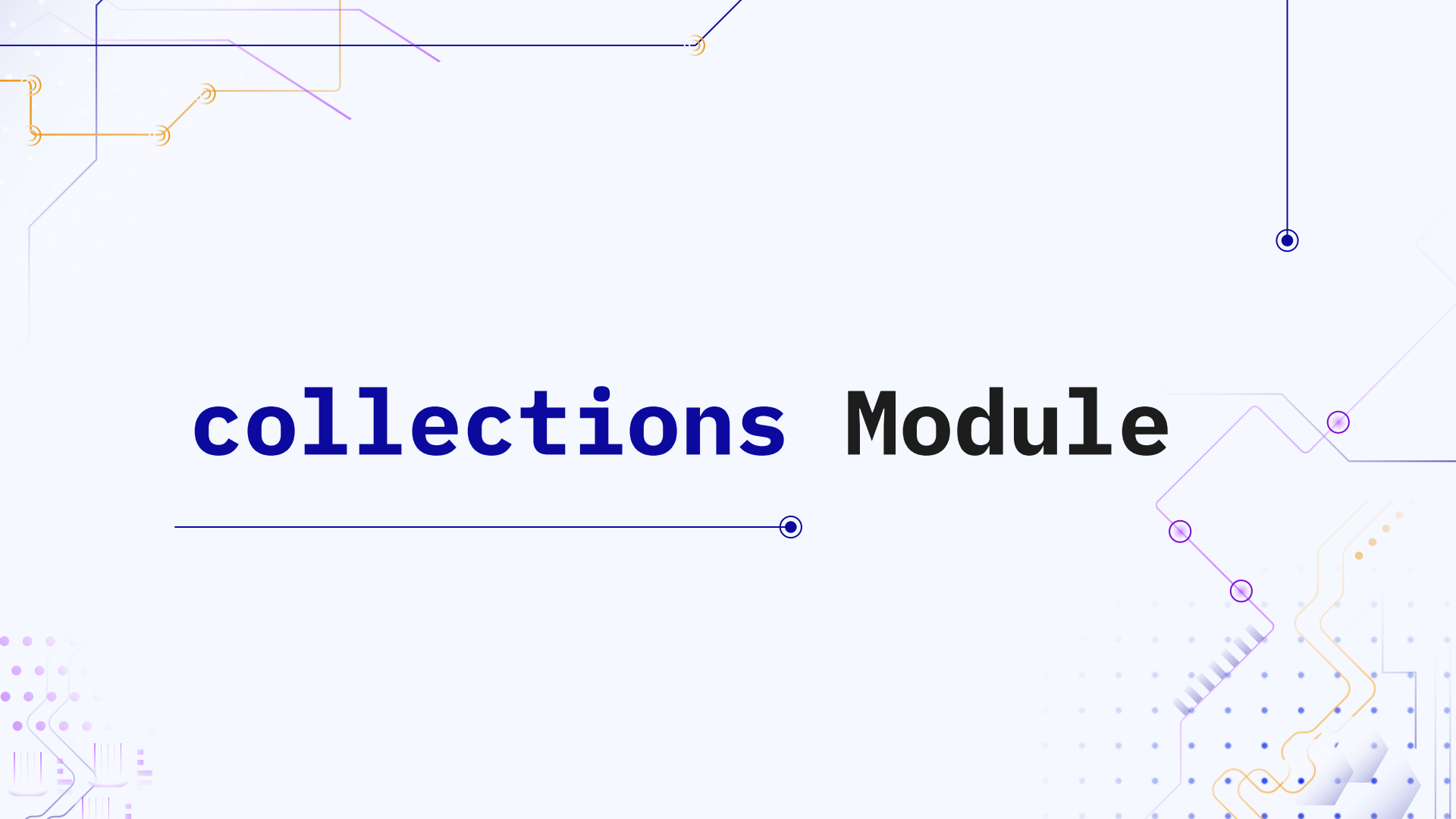




collections Module

Introduction

collections adds “**smart containers**” that solve common problems like:

- Counting items easily
- Making fast queues/stacks
- Handling missing dictionary keys gracefully
- Keeping only the last N items
- Having dictionary items with attribute-style access

Syntax

```
import collections
from collections import Counter, deque, defaultdict, namedtuple, OrderedDict
```

Counter

What it is

A **Counter** is like a dictionary where:

- keys = items
- values = how many times they appear

When to use

- Frequency counting (words, characters, votes, items)
- Finding most common items

Python

```
from collections import Counter

data = ["a", "b", "a", "c", "a", "b"]
c = Counter(data)

print(c)                # Counter({'a': 3, 'b': 2, 'c': 1})
print(c["a"])           # 3
print(c["z"])           # 0 (no KeyError!)
```

Counter

Most common

```
print(c.most_common(2)) # [('a', 3), ('b', 2)]
```

Update counts

```
c.update(["a", "d", "d"])  
print(c) # Counter({'a': 4, 'b': 2, 'd': 2, 'c': 1})
```

Convert to normal dict

```
normal = dict(c) # {'a': 4, 'b': 2, 'c': 1, 'd': 2}
```

Count characters/words

```
Counter("banana") # {'a': 3, 'n': 2, 'b': 1}  
Counter("i love love python".split()) # {'love': 2, 'i': 1, 'python': 1}
```

deque

What it is

deque (“deck”) is a double-ended queue:

- super fast append and popleft
- perfect for queues, BFS, sliding window

Why not list?

- **list.pop(0)** is slow because everything shifts left.
- **deque.popleft()** is fast.

deque

Python

```
from collections import deque

dq = deque([1, 2, 3])
dq.append(4)      # add right
dq.appendleft(0)  # add left

print(dq)         # deque([0, 1, 2, 3, 4])

dq.pop()          # remove right
dq.popleft()      # remove left
```

deque

Rotate

```
from collections import deque

dq = deque([1, 2, 3, 4])
dq.rotate(1)      # shift right by 1 -> last comes first
print(dq)         # deque([4, 1, 2, 3])
dq.rotate(-2)     # shift left by 2
```

Fixed size deque → gives only the latest N items

```
dq = deque(maxlen = 3)
dq.extend([10, 20, 30])
dq.append(40)
print(dq)         # deque([20, 30, 40], maxlen=3)
```

Use cases: recent history, last 50 messages/logs, sliding window.

defaultdict

What it is

A **defaultdict** is a **dict** that creates a default value automatically if the key doesn't exist.

Python

```
from collections import defaultdict

d = defaultdict(int) # int() -> 0
d["a"] += 1
d["b"] += 5
print(d)              # {'a': 1, 'b': 5}
```

Common defaults

- `defaultdict(int)` → 0 (counting)
- `defaultdict(list)` → [] (grouping)
- `defaultdict(set)` → `set()` (unique grouping)

Caution:

defaultdict reduces boilerplate, but reading a missing key creates it.

namedtuple

What it is

A **namedtuple** is like a **tuple**, but you can access values using names, not only indexes.

Python

```
from collections import namedtuple

d = namedtuple("Person", ["name", "age"])
p = d("Aman", 21)
print(p.name)           # Aman
print(p.age)            # 21
```

Important points

- Immutable (cannot change fields like normal tuple)
- Lightweight, readable

ChainMap

What it is

ChainMap lets you treat multiple **dicts** like one combined mapping without merging.

Python

```
from collections import ChainMap

defaults = {"theme": "light", "lang": "en"}
user = {"theme": "dark"}

cfg = ChainMap(user, defaults)
print(cfg["theme"])      # dark
print(cfg["lang"])       # en
```

Remember: In ChainMap, the first mapping wins.



Practice Set – 3

Download Link in **Description** and **Pinned Comment**

WATCH

Level up your coding with each episode in this focused Python series.



Next Video!

**Practice Set – 3
Solution**

